

Ontology Builder

Executive Product Brief

Product Name: Ontology Builder **Platform:** AI-Assisted Palantir Foundry–style Data Ontology Design
Version: 1.0 Prototype **Date:** July 2026

1. Executive Summary

Ontology Builder is an AI-powered platform that turns a plain-language description of an organization into a complete, queryable **data ontology** — object types, properties, relationships, actions, permissions, validation rules, and lifecycle state machines — modeled in the style of a Palantir Foundry ontology. A business user answers a short guided questionnaire; a multi-phase AI agent pipeline then researches the domain, designs the ontology, generates the actions and governance rules, validates and self-repairs the model, writes the ontology as versioned YAML, and provisions a working SQLite database seeded with realistic synthetic data. The user can then explore the result as an interactive **network graph**, browse the generated YAML, query the data in natural language, and **request changes** that are applied surgically and safely — all from a single, professional workspace.

At the heart of the platform is a **Microsoft Agent Framework (MAF) workflow** that orchestrates a team of specialist agents through a fixed pipeline, streaming its progress to the UI over the **AG-UI protocol** (SSE). The interface is not a chatbot with a hidden backend — every phase of the AI's reasoning is visible in real time: a phase timeline advances step by step, an agent-activity feed shows which specialist and sub-agent is running, a research trace exposes what the AI found, and an approval dialog gates the one irreversible step (writing to the database) behind explicit human consent.

The same machinery powers two flows from one design: **Build** (create an ontology from scratch) and **Update** (modify an existing ontology in place). The update flow applies changes as surgical patches and migrates the database **additively** — new tables and columns are added, existing data is never dropped.

2. The Problem

Designing a data ontology — the semantic layer that turns raw tables into meaningful, connected business objects — is slow, specialized, and inconsistent:

Challenge	Impact
Ontology design requires scarce expertise — object modeling, relationships, permissions, lifecycles	Only data architects can do it; business teams wait weeks for a first draft
Blank-page problem — nothing to react to at the start	Projects stall before the first object type is defined
Design ↔ database drift — the model and the actual schema diverge	The ontology becomes documentation, not a source of truth
No safe way to iterate — changing an ontology risks breaking downstream data	Teams freeze the model rather than evolve it, or lose data on rebuilds
Quality is invisible — isolated object types, incomplete relationships slip through	The graph looks connected but has silent gaps

Opaque AI — "generate my ontology" black boxes
produce output no one trusts

Users can't see or steer the reasoning, so
they don't adopt it

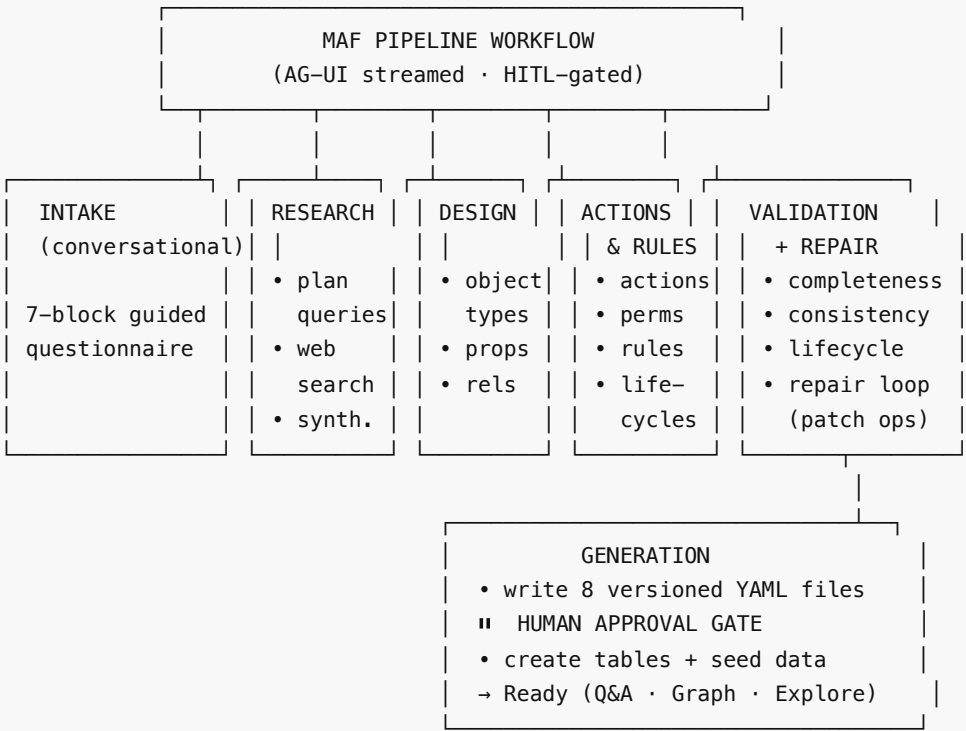
3. The AI Engine — A Multi-Phase MAF Pipeline

3.1 Design Philosophy

Ontology Builder is built on the principle that **the AI should drive the workflow and show its work**. Each phase is an autonomous specialist; the UI is a live visualization layer over the pipeline's streamed events. When the timeline shows "Design," the Ontology Designer's sub-agents are actually running. When the graph appears, it is rendered from the exact object types and relationships the agents produced. Nothing is faked, and the one irreversible action always asks first.

3.2 Pipeline Architecture

The build pipeline is a deterministic MAF **Workflow** — one executor per phase, chained in a fixed order, each delegating to a specialist that orchestrates fine-grained sub-agents:



3.3 Specialists & Sub-Agents

Each phase specialist decomposes into independent sub-agents, each a focused prompt + schema:

Specialist	Sub-Agents	Output
Intake	Clarifier, Refiner, Emitter	Structured requirements (company, domain, users, workflows, constraints)

Research	Search Planner, Web Researcher, Domain Synthesizer	Industry patterns, recommended object types & relationships, best practices
Ontology Designer	Object-Type Agent, Property Agent, Relationship Agent	Object types with properties + a connected relationship graph
Actions & Rules	Action Designer, Permission Designer, Validation Designer, Lifecycle Designer	Actions, role-based permissions, validation rules, lifecycle state machines
Validator	Completeness Checker, Consistency Checker, Lifecycle Validator, Repair Agent	Scored validation report + auto-repaired ontology
Generation	YAML Writer, Datastore Provisioner, Seed Generator	8 YAML files, DB tables, synthetic rows, suggested questions

3.4 Visible Agent Activity

The workspace renders the pipeline's streamed AG-UI events in real time:

- **Phase timeline** (top) — Intake → Research → Design → Actions & Rules → Validation → Generate → Ready, each step showing pending / active (spinner) / done (check).
- **Agent Activity feed** — live cards for each running specialist and sub-agent ("Domain researcher — running", "Ontology designer — done").
- **Research Trace** — the industry/domain detected, recommended object types as chips, and industry patterns the research surfaced.
- **Agent Skills panel** — the advertised skill catalog, with the skill relevant to the current phase highlighted.

Every executor transition streams as a `STEP_STARTED` / `STEP_FINISHED` event; custom events carry phase status, agent activity, the research trace, and the validation report.

3.5 Human-in-the-Loop Approval

The pipeline separates the safe step (writing YAML files) from the irreversible step (provisioning and seeding the database). Before the database is touched, the workflow **pauses** and emits an interrupt; the UI shows an approval dialog listing the tables to be created. Only on explicit approval does provisioning run — and it can be declined, leaving the ontology files intact without a database.

3.6 Self-Repairing Quality Loop

Validation is deterministic, not vibes. The Completeness, Consistency, and Lifecycle checkers flag concrete problems — including **isolated object types** (no relationships) and **incomplete relationships** (missing cardinality or foreign key). Issues feed a **Repair Agent** that plans surgical patch operations (add a relationship to connect an island, complete a foreign key, fix a lifecycle) which are applied programmatically. The loop repairs and re-validates — even actionable warnings are auto-repaired — until the model meets a quality threshold, then proceeds.

4. The Update Flow — Safe, In-Place Evolution

An ontology is never "done." The Update flow lets a user evolve an existing ontology without rebuilding it or losing data. It reuses the entire pipeline, but operates as a **delta** against the current model.

4.1 Batch Change Collection

From the Ontology workspace, **"Request changes"** opens a modal where the user adds change requests one by one into a running list ("Add a priority field to Incident", "Add a SafetyIncident object type", ...), reviews and edits them, then approves the whole batch at once — avoiding a separate round-trip per change.

4.2 Adaptive Follow-Ups → Confirm → Apply

Batch of changes → Adaptive follow-up questions (per change) → Summary → Confirm
→ Update pipeline: design-patch → actions/rules → validation → generation

1. **Update-Intake Specialist** asks tailored follow-up questions per change, grounded in the *existing* ontology (real object type / property names), auto-advancing through the batch and then summarizing.
2. On confirmation, the **Update Designer** plans surgical **patch operations** (add/modify/remove object types, properties, relationships) applied by a shared patch engine — the ontology is mutated in place, not regenerated.
3. Actions/rules and validation re-run over the merged model (with the same connectivity + completeness guardrails), and generation writes a **version-bumped** ontology (1.0 → 1.1 → ...).

4.3 Additive, Data-Safe Database Migration

The database already holds data, so migration is strictly additive and gated behind the same approval dialog:

- **New tables** → created and seeded.
- **New columns** on existing tables → `ALTER TABLE ADD COLUMN (nullable)` — existing rows preserved, no re-seeding.
- **Removed** items in the ontology are **not dropped** from the database — data is never destroyed; they simply leave the model, graph, and YAML.

5. Exploring the Ontology

Once an ontology is ready, it lives in a single **Ontology workspace** with three tabs, plus dedicated Q&A and Data views:

5.1 Interactive Network Graph

The ontology rendered as a live graph (React Flow): object types are node cards (icon, property count), relationships are directed, labeled edges showing cardinality. Users pan/zoom, auto-layout, click a node to inspect its properties and relationships in a side panel, or click an edge to see its cardinality and foreign key. Isolated types and relationships with unknown endpoints are surfaced as a warning, making quality gaps visible.

5.2 YAML Viewer

The eight generated files — `object_types`, `properties`, `relationships`, `actions`, `permissions`, `validation_rules`, `lifecycle_states`, `data_mapping` — with syntax highlighting, so the model is fully inspectable and portable.

5.3 Test Q&A (Natural-Language → SQL)

A **skills-enabled MAF agent** translates plain-language questions ("What's the status distribution of support cases?") into safe, read-only SQL against the provisioned database, executes it, and returns a formatted answer with the generated SQL, a summary table, key insights, and suggested follow-up questions.

5.4 Data Explorer

Browse the actual seeded tables and rows, confirming the ontology maps to a real, queryable schema.

6. Agent Skills — Progressive-Disclosure Knowledge

The platform ships a catalog of **9 agent skills** (agentskills.io-style `SKILL.md` packages) encoding the domain conventions each phase should follow. Skills are surfaced in the UI (the panel highlights the skill relevant to the active phase, including update phases) and, crucially, **influence the agents**:

- **Most specialists** have the relevant skill's guidance injected into their system prompt per phase — so the skill actually shapes generation, validation, and update output.
- **The Q&A agent** is a true MAF Agent with a `SkillsProvider` attached and read-only skill tools auto-approved, demonstrating genuine progressive disclosure (the model calls `load_skill('nl-to-sql')` on demand).

Skill	Phase(s)	Covers
ontology-design-patterns	design, update_design	Object type / property / relationship conventions
graph-connectivity	design, update_design	No isolated types, complete relationships, cardinality/direction
actions-and-permissions	actions_rules, update_actions_rules	Actions, role-based permissions, validation, lifecycles
validation-heuristics	validation, update_validation	Completeness/consistency checks + repair guidance
ontology-update-management	all update_* phases	Change analysis, surgical patches, additive migration
requirements-intake	intake	Structured requirements gathering
domain-research	research, update_research	Domain research + research-trace usage
generation-provisioning	generating, update_generating	YAML emission, DDL, seeding, approval gate
nl-to-sql	qa	Safe NL→SQL translation rules

7. Stakeholders & User Journeys

7.1 Business / Domain User (Primary)

Journey: Build an ontology from scratch

1. Opens Build, answers the 7-block questionnaire (company profile, domain, problem, data sources, users, workflows, constraints) — with tailored multiple-choice options.
2. On completion the pipeline auto-starts; the phase timeline advances through Research → Design → Actions & Rules → Validation while the Agent Activity feed and Research Trace stream live.
3. At Generation, an approval dialog lists the tables to be created; the user approves.
4. Lands in the Ontology workspace — explores the graph, reviews YAML, and asks the data questions the AI suggested.

Journey: Evolve the ontology

1. While exploring, clicks **Request changes**, adds several changes to the batch, edits, and approves.
2. Answers a few tailored follow-ups per change; reviews the summary; confirms.
3. Watches the update pipeline apply surgical patches, re-validate, and (on approval) migrate the database additively — existing data preserved.
4. The graph updates to show the new/changed object types and relationships.

7.2 Data Architect / Reviewer

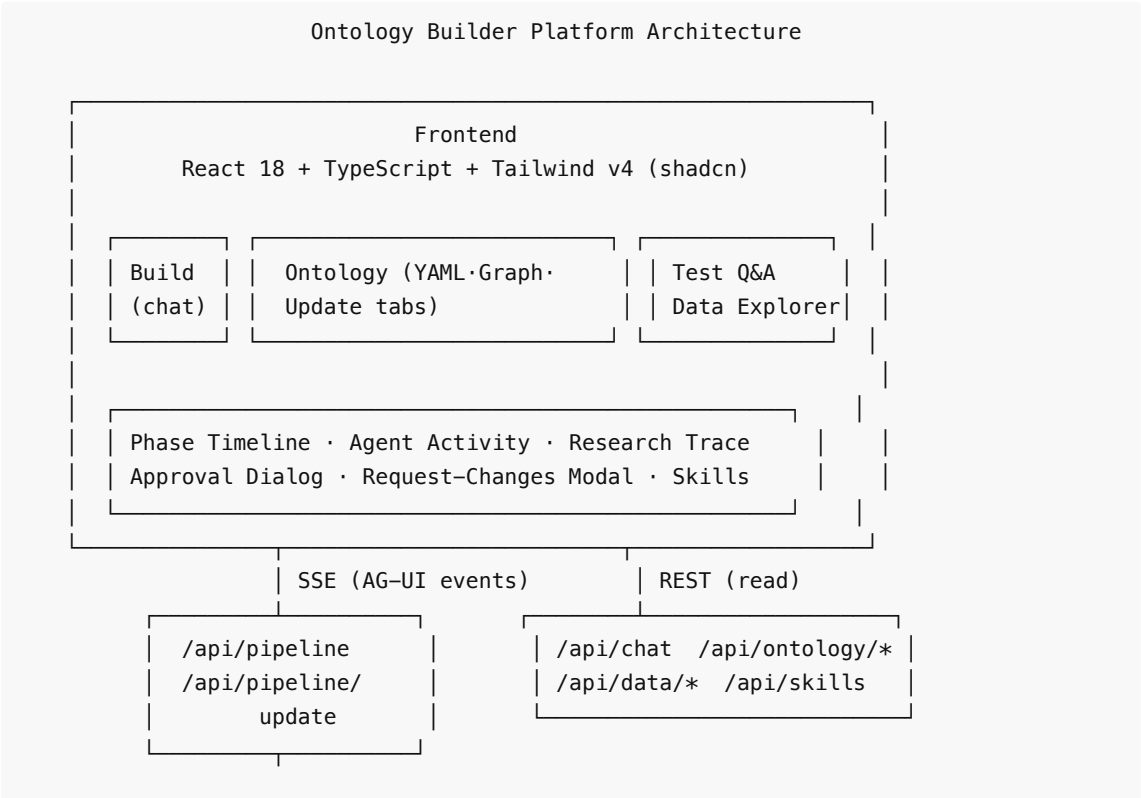
Reviews the generated YAML and graph as a first draft to refine, trusts the validation report's connectivity and completeness guarantees, and uses the update flow to iterate safely rather than hand-editing schema.

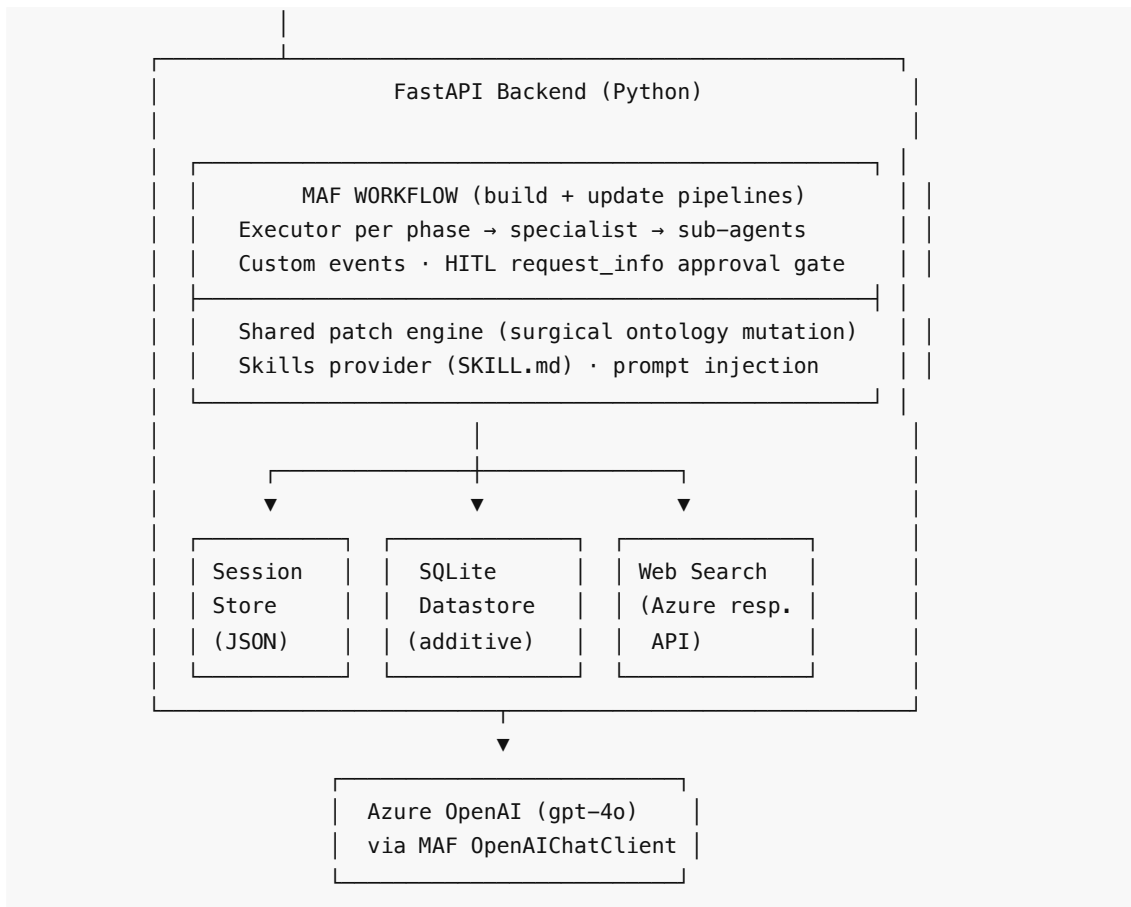
7.3 Analyst / Consumer

Uses Test Q&A to interrogate the seeded data in natural language, validating that the ontology answers real business questions before it is adopted.

8. Technical Architecture

8.1 Architecture Overview





8.2 Technology Stack

Layer	Technology	Rationale
Frontend Framework	React 18 + TypeScript + Vite	Fast, strongly-typed SPA
Design System	Tailwind CSS v4 (CSS-first OKLCH tokens) + shadcn-style primitives	Professional, light/dark, consistent
Graph Visualization	React Flow (@xyflow/react) + Dagre layout	Custom node cards, labeled edges, auto-layout
Markdown / Icons	react-markdown · lucide-react	Chat rendering, consistent iconography
Backend	FastAPI (Python, async)	Ideal for streaming AI workloads
Agent Framework	Microsoft Agent Framework (MAF) 1.11	Workflow orchestration, agents, skills
Protocol	AG-UI (SSE)	Real-time streaming of agent lifecycle to the UI
AI Model	Azure OpenAI gpt-4o (via MAF OpenAIChatCompletionClient, key auth)	Enterprise-grade, structured JSON output

Datastore	SQLite (aiosqlite)	Zero-config provisioning + seeding; additive ALTER migration
Skills	SKILL.md packages + SkillsProvider	Progressive-disclosure domain knowledge
Web Search	Azure OpenAI Responses API (web_search tool)	Domain research during the research phase

8.3 Key Technical Decisions

Deterministic Workflow, not free-form orchestration. The pipeline is a fixed, linear MAF Workflow (executor per phase), which matches the ontology-build process and makes progress predictable and streamable — rather than an intent-routed agent mesh.

AG-UI streaming for a transparent UI. The workflow is exposed over the AG-UI SSE protocol; executor lifecycle and custom events drive the phase timeline, agent-activity feed, research trace, and approval interrupts — so the AI's reasoning is observable, not hidden.

One pipeline, two flows (build + update). The update flow reuses the same specialists and event scaffolding but operates as a delta via a **shared patch-op engine** — the same mechanism the validator's Repair Agent uses — keeping mutation surgical and consistent across build-repair and user-driven updates.

Additive, data-safe database migration. Updates create new tables/columns via ALTER TABLE ADD COLUMN and never drop existing data; the ontology version is bumped with an extends reference to the prior version.

Human-in-the-loop on the only irreversible step. Database provisioning is gated behind a MAF request_info interrupt surfaced as an approval dialog — the user always consents before data is written.

Skills that actually shape output. Rather than decorative labels, skill bodies are injected into specialist prompts per phase, and the Q&A agent uses a real SkillsProvider with load_skill — demonstrating both practical and progressive-disclosure skill usage.

9. Pipeline Phases & Endpoints

Phase	What Happens	Surfaced As
intake	7-block guided questionnaire → structured requirements	Build chat + phase step 1
research	Plan queries → web search → synthesize domain knowledge	Research Trace panel
design	Object types → properties → relationships (connected graph)	Agent Activity
actions_rules	Actions, permissions, validation rules, lifecycles	Agent Activity
validation	Deterministic checks + auto-repair loop → scored report	Agent Activity
generating	Write YAML → approve → create tables + seed data	Approval Dialog

qa	Ontology ready — explore, query, update	Ontology / Q&A / Data views
update_*	Mirror of the above for in-place, additive changes	Update tab (same UI)

Key endpoints: POST /api/pipeline and POST /api/pipeline/update (AG-UI SSE); POST /api/chat (intake + update-intake); POST /api/ontology/{id}/ask (Q&A); GET /api/ontology/{id}/graph and /files (structure); GET /api/data/{id}/tables (data); GET /api/skills (skill catalog).

10. Expected Outcomes

Efficiency Gains

Metric	Traditional	With Ontology Builder	Improvement
First-draft ontology	Days–weeks (architect-led)	Minutes (guided pipeline)	Orders of magnitude
Model ↔ database alignment	Manual, drift-prone	Generated together, in sync	Single source of truth
Iterating on the model	Risky rebuild / frozen	Surgical, additive, data-safe	Evolve without loss
Validating the design	Ad hoc review	Deterministic checks + auto-repair	Objective quality gate
Testing the model	Write SQL by hand	Ask in natural language	Instant validation
Trust in AI output	Black box	Streamed, step-by-step, approval-gated	Transparent & steerable

Strategic Impact

- **Democratizes ontology design** — business users produce a credible first draft without an architect.
- **Keeps model and data in sync** — the ontology *is* the schema, generated and provisioned together.
- **Safe evolution** — additive updates mean the ontology is a living model, not a frozen artifact.
- **Trustworthy AI** — every phase is visible and the irreversible step is consented, driving adoption.
- **Portable output** — standard YAML + a real queryable database, not a proprietary black box.

11. Security & Compliance Considerations

Area	Current State	Production Recommendation
Authentication	Not implemented (prototype)	Azure AD / OAuth 2.0 SSO
Authorization	Single user	Role-based access (Designer, Reviewer, Consumer)

Secrets	.env (git-ignored), Azure key auth	Key Vault / managed identity
Datastore	Shared SQLite file	Per-tenant Postgres; row-level security
Q&A safety	SELECT-only enforced	Query allow-listing, statement timeouts
HITL approval	DB provisioning gated	Extend approval to all schema-mutating steps + audit log
Audit trail	Session store (JSON)	Full audit of pipeline runs, approvals, and data access

12. Roadmap to Production

Phase 1: Datastore & Scale

- Swap SQLite for PostgreSQL (backend abstraction already exists); real migration manager with rollback.
- Full ALTER/rename support and safe destructive-change workflows behind explicit confirmation.

Phase 2: Governance & Auth

- Azure AD SSO, role-based access, audit logging of every pipeline run and approval.
- Ontology registry with versioning, diffing, and extension validation across versions.

Phase 3: Deeper Agentic Capabilities

- Wire the `SkillsProvider` into more specialists (beyond Q&A) for full progressive disclosure.
- Richer research (multi-source, cited) and RAG over an organization's existing schemas/policies.
- Export ontology to real Foundry / other ontology formats.

Phase 4: Collaboration & Deployment

- Multi-user sessions, shared ontologies, review/approve workflows.
- Docker/compose hardening, observability (OpenTelemetry is available in MAF), and performance tuning.

13. Technology Summary

Frontend:	React 18 · TypeScript · Vite · Tailwind v4 (OKLCH tokens) · shadcn-style UI
Graph:	React Flow (@xyflow/react) · Dagre layout · custom node cards + labeled edges
AI Engine:	MAF Workflow · specialist agents + sub-agents · 9 agent skills · self-repair loop
Backend:	FastAPI · Python · Microsoft Agent Framework 1.11 · AG-UI (SSE) protocol
AI Model:	Azure OpenAI gpt-4o via MAF OpenAIChatCompletionClient (key auth)
Datastore:	SQLite (aiosqlite) · generated DDL + synthetic seed · additive ALTER migration
Flows:	Build (from scratch) + Update (surgical, additive, data-safe, versioned)

HITL: request_info approval gate before database provisioning
UX: Phase Timeline · Agent Activity · Research Trace · Approval Dialog · Skills Panel · light/dark
Endpoints: /api/pipeline · /api/pipeline/update · /api/chat · /api/ontology/* · /api/data/* · /api/skills

*Ontology Builder — from a plain-language brief to a validated, queryable data ontology in minutes.
Transparent by design — every phase visible, every irreversible step approved. Built AI-first on the Microsoft Agent Framework.*